



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

**ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №11
«ВИЗУАЛИЗАЦИЯ И РАБОТА
С БОЛЬШИМИ ДАННЫМИ»**

по дисциплине «Языки программирования для статистической обработки
данных»

Студент группы

ИНБО-20-23. Боргачев Т.М.

(подпись)

Преподаватель

Трушин СМ

(подпись)

Москва 2025 г.

ВВЕДЕНИЕ

Цель работы: освоить методы интерактивной визуализации и обработки больших временных данных.

Задачи:

1. Построить интерактивные графики продаж с помощью Plotly и Shiny.
2. Оптимизировать обработку данных с использованием pandas/dask и data.table.

Данные:

- Датасет: **train_time_series.csv** (временные ряды продаж).

РЕЗУЛЬТАТЫ РАБОТЫ

Интерактивная визуализация

Код для интерактивной визуализации временных рядов с помощью plotly представлен в листинге 1.

Листинг 1 - Интерактивный график продаж (Python)

```
import plotly.express as px
import pandas as pd
import time

# Загрузка данных
df = pd.read_csv("train_time_series.csv")

# Замер времени для построения графика
start_time = time.time()

# График временного ряда
fig = px.line(df, x='date', y='sales', color='family',
              title="Продажи по категориям товаров")
fig.update_xaxes(rangeslider_visible=True)

# Показать график
fig.show()

# Завершение замера времени
end_time = time.time()
print(f"Время выполнения построения интерактивного графика: {end_time - start_time:.2f} секунд")
print("График успешно построен. Интерактивные функции включают зумирование и фильтрацию по категориям.")
```

Код для интерактивной визуализации временных рядов с помощью shiny представлен в листинге 2.

Листинг 2 – Интерактивный график продаж (R)

```
library(shiny)
library(ggplot2)

# Загрузка данных
df <- read.csv("train_time_series.csv")

# Проверка, что данные загружены корректно
if (!"family" %in% colnames(df)) {
  stop("В данных отсутствует столбец 'family'. Проверьте файл train_time_series.csv.")
}
if (!"date" %in% colnames(df)) {
  stop("В данных отсутствует столбец 'date'. Проверьте файл train_time_series.csv.")
}
if (!"sales" %in% colnames(df)) {
  stop("В данных отсутствует столбец 'sales'. Проверьте файл train_time_series.csv.")
}

# Преобразование столбца date в формат даты
df$date <- as.Date(df$date)

# UI
ui <- fluidPage(
  titlePanel("Анализ продаж"),
  sidebarLayout(
    sidebarPanel(
      selectInput("family", "Выберите категорию:",
        choices = unique(df$family))
    ),
    mainPanel(
      plotOutput("salesPlot") # Используем plotOutput для ggplot2
    )
  )
)

server <- function(input, output) {
  output$salesPlot <- renderPlot({
    # Фильтрация данных по выбранной категории
    df_filtered <- subset(df, family == input$family)

    # Построение графика
    ggplot(df_filtered, aes(x = date, y = sales)) +
      geom_line(color = "blue") +
      labs(title = paste("Продажи:", input$family),
        x = "Дата",
        y = "Продажи") +
      theme_minimal()
  })
}

shinyApp(ui, server)
```

Пример построения интерактивного графика на Python представлен на рис. 1.

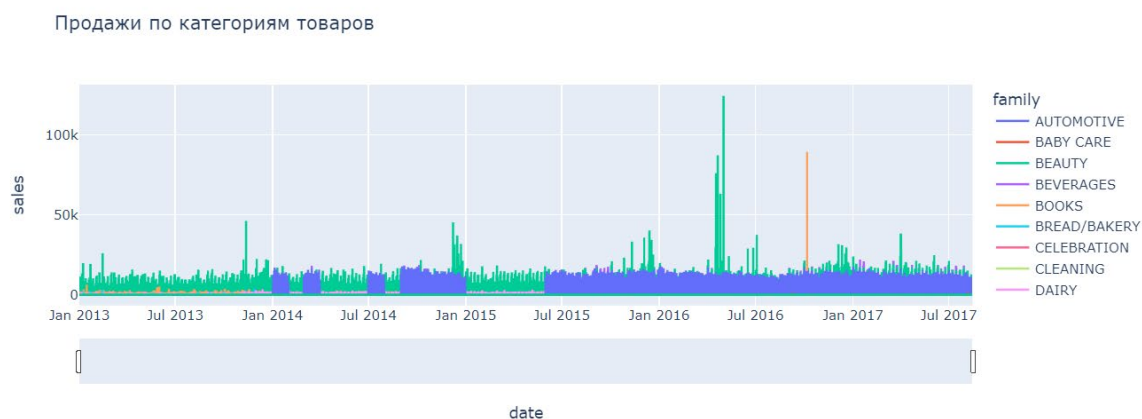


Рисунок 1 – Визуализация графика Python

Пример построения интерактивного графика на R представлен на рис. 2.

Анализ продаж

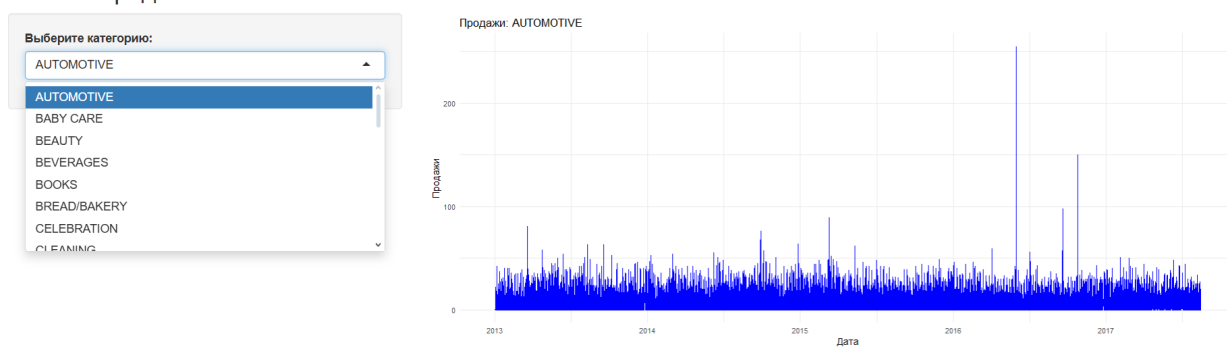


Рисунок 2 – Визуализация графика R

Работа с большими данными

Код для агрегации больших данных с помощью dask представлен в листинге 3.

Листинг 3 – Агрегация данных (Dask)

```
import dask.dataframe as dd
import time

# Загрузка данных с использованием Dask
ddf = dd.read_csv("train_time_series.csv")

# Замер времени для агрегации
start_time = time.time()

# Агрегация данных
monthly_sales = ddf.groupby(['family', 'date'])['sales'].sum().compute()

# Завершение замера времени
end_time = time.time()
print(f"Время выполнения агрегации с Dask: {end_time - start_time:.2f} секунд")
print("Агрегация завершена. Результат содержит суммарные продажи по категориям и датам.")

# Вывод первых строк результата
print("Первые строки агрегированных данных:")
print(monthly_sales.head(100))
```

Код для агрегации больших данных с помощью `data_table` представлен в листинге 4.

Листинг 4 – Агрегация данных (`data_table`)

```
library(data.table)

# Загрузка данных с использованием data.table
start_time <- Sys.time()

dt <- fread("train_time_series.csv")

# Агрегация данных
monthly_sales <- dt[, .(total_sales = sum(sales)), by = .(family, date)]

# Завершение замера времени
end_time <- Sys.time()

# Вывод времени выполнения агрегации
print(paste("Время выполнения агрегации с data.table:",
            as.numeric(difftime(end_time, start_time, units = "secs")),
            "секунд"))

# Вывод первых строк результата
print("Первые строки агрегированных данных:")
print(head(monthly_sales))
```

Результат агрегации в Python представлен на рис. 3.

```

Время выполнения агрегации с Dask: 2.06 секунд
Агрегация завершена. Результат содержит суммарные продажи по категориям и датам.
Первые строки агрегированных данных:
family      date
AUTOMOTIVE  2013-01-01      0.0
            2013-01-02    255.0
            2013-01-03    161.0
            2013-01-04    169.0
            2013-01-05    342.0
            ...
            2013-04-06    341.0
            2013-04-07    345.0
            2013-04-08    223.0
            2013-04-09    196.0
            2013-04-10    199.0
Name: sales, Length: 100, dtype: float64

```

Рисунок 3 – Агрегация больших данных (Python)

Результат агрегации в R представлен на рис. 4.

```

[1] "Время выполнения агрегации с data.table: 0.470329999923706 секунд"
[1] "Первые строки агрегированных данных:"
      family      date total_sales
      <char>    <IDat>      <num>
1:  AUTOMOTIVE 2013-01-01      0.000
2:   BABY CARE 2013-01-01      0.000
3:    BEAUTY   2013-01-01      2.000
4:  BEVERAGES 2013-01-01    810.000
5:    BOOKS    2013-01-01      0.000
6: BREAD/BAKERY 2013-01-01    180.589

```

Рисунок 4 – Агрегация больших данных (R)

ЗАКЛЮЧЕНИЕ

В рамках практической работы были освоены методы интерактивной визуализации и обработки больших временных данных с использованием языков программирования Python и R. В ходе выполнения задач были успешно построены интерактивные графики продаж с помощью библиотек Plotly (Python) и Shiny (R). Данные инструменты позволили создать удобные и информативные визуализации, которые могут быть использованы для анализа динамики продаж в зависимости от категории товаров.

Также была проведена работа по агрегации больших данных с использованием библиотек Dask (Python) и data.table (R). Эти инструменты показали высокую эффективность при работе с большими объемами данных, обеспечивая быструю обработку и агрегацию информации.

В результате выполнения работы были достигнуты следующие ключевые результаты:

1. Созданы интерактивные приложения для анализа временных рядов продаж.
2. Освоены методы оптимизации обработки данных с использованием современных технологий.
3. Получен практический опыт применения различных инструментов для работы с большими данными.

Выполненная работа подтвердила важность использования специализированных библиотек и фреймворков для эффективного анализа данных. Полученные навыки могут быть применены для решения реальных задач бизнес-аналитики, прогнозирования и принятия решений на основе данных.